

Chasing Zestimates: A Machine Learning Challenge in Queens Real Estate

Final project for Math 342W Data Science at Queens College

May 26th, 2026

By Danielle Collins

In collaboration with:

Rowida Zaid

Abstract

Behind every apartment sale in New York City lies a complex interplay of economic and neighborhood factors that makes precise price prediction both difficult and valuable. Despite the abundance of real estate data, reliably predicting apartment sale prices in dense urban markets continues to be a nontrivial machine learning problem. This study develops a predictive model for apartment sale prices in Queens, New York using a dataset of 518 co-op and condominium transactions from February 2016 to February 2017, sourced from the Multiple Listing Service of Long Island via Amazon Mechanical Turk. Three models—ordinary least squares (OLS), a regression tree, and a tuned random forest (500 trees, $mtry = 13$)—were trained and evaluated using 5-fold cross-validation. The dataset, originally comprising 2,230 observations, required substantial cleaning and feature engineering due to extensive missingness and reduction in usable records. Feature construction incorporated neighborhood-level variables, including mean household income by ZIP code, crime rates, Consumer Price Index at time of sale, Queens neighborhood regions, and binary indicators for missing data in heavily incomplete variables. Missingness in continuous and categorical features was addressed using missForest, a random-forest-based imputation method that leverages all available features to produce observation-specific imputed values rather than applying a single global statistic. The random forest model achieved the strongest out-of-sample performance, with an RMSE of \$79,818, normalized RMSE of 0.2559, and R^2 of 0.7939, outperforming both the linear model (RMSE \$81,510; normalized RMSE 0.2614; R^2 0.7831) and the regression tree (RMSE \$106,073; normalized RMSE 0.3401; R^2 0.6350). While the model demonstrates strong predictive performance for Queens apartment pricing, it does not reach Zillow’s reported median error rate of 2.3%, a gap likely driven by differences in data scale, proprietary feature engineering, and broader market coverage rather than modeling approach.

1. Introduction

Real estate pricing is one of the most consequential prediction problems in applied data science. For buyers, sellers, and investors alike, accurately estimating the market value of a property can mean the difference between a sound financial decision and a costly mistake.

As Harold Samuel, founder of one of the largest property development and investment companies in the United Kingdom (Land Securities Group), famously stated, “The three most important factors in buying a home are location, location, and location.” This principle motivates much of modern real estate prediction modeling, cementing the ideology that true signal can be found in a wide variety of factors. In determining not only the structural and physical integrity of the properties but analyzing the geographic and neighborhood-level variables associated with them, we reap a beautiful plethora of some of the strongest predictors of housing prices.

This project adopts a supervised machine learning pipeline to predict apartment sale prices in Queens, New York — a borough characterized by extraordinary neighborhood diversity, ranging from the high-density commercial corridors of Long Island City to the quiet residential streets of Bayside and Little Neck. In other words, an ideal point of study for our project.

Here, the unit of observation is an individual apartment sale, specifically focused on co-ops and condominiums sold between February 2016 and February 2017 in mainland Queens zip codes, with sale prices capped at \$1,000,000. The response variable is the recorded sale price in dollars (\$). The dataset, sourced from the Multiple Listing Service of Long Island via Amazon Mechanical Turk, contains 2,230 raw observations, of which only 518 survive filtering, cleaning, and duplicate removal.

Three predictive models are developed and contrasted: a single regression tree (YARF-CART), an Ordinary Least Squares linear regression, and a random forest (YARF) tuned via 5-fold cross validation (Kapelner, 2024). All models are evaluated using out-of-sample RMSE, normalized RMSE, and R^2 via 5-fold cross validation, ensuring that reported performance metrics reflect genuine generalization to unseen data rather than in-sample fit. The random forest achieves the strongest out-of-sample performance, explaining approximately 79% of variance in sale price with a cross-validated RMSE of \$79,818 and a normalized RMSE of 0.2559 — meaning the typical prediction error is about 25.6% of the mean sale price of \$311,877. A natural benchmark for this work is Zillow’s proprietary Zestimate algorithm, which reports a median error of 2.3% for the NYC area (Zillow, 2020). As discussed in the performance and discussion sections, closing this gap would require fundamentally different

data resources rather than improved modeling alone.

2. The Data

To reiterate, the raw dataset consisted of 2,230 observations obtained from the Multiple Listing Service of Long Island (MLSI), a database of residential real estate listings and transactions. The information was harvested through Amazon Mechanical Turk, where workers manually extracted and recorded apartment characteristics, financial attributes, and sale information from MLSI listings into a structured dataset. The resulting data is broadly representative of co-op and condominium apartment sales in Queens, New York during the 2016–2017 time period.

Thus, each observation corresponds to a single property listing. Also, the population of interest is defined as co-ops and condominiums sold in mainland Queens between February 2016 and February 2017 at prices up to \$1,000,000, restricted to the zip codes specified in Table 1 of the project specification. After filtering to this population, removing 1,702 observations with missing sale prices, and eliminating 3 true duplicate entries — records sharing identical address, sale price, and sale date — the final modeling dataset contains 518 observations.

The dataset is reasonably representative of the defined population, though several caveats apply. Mainly, the reoccurring issue was that the MTurk collection methodology was severely susceptible to noise. Due to workers manually transcribing listing data, the undesired outcome was numerous typographical errors (e.g. “eys” instead of “yes”), inconsistent categorical encodings (e.g. “other” vs. “Other”), and substantial missingness across several features (e.g. `common_charges` — 390 NAs out of 518 (75%)).

Ultimately, although the dataset provides substantial coverage of co-op and condominium apartment sales across Queens, it likely underrepresents both luxury properties and more recent market conditions due to the imposed \$1 million sale-price cap and the crowdsourced data collection process. Additionally, several notable outliers are also present within the dataset. For example, the maximum recorded square footage of 6,215 square feet is substantially larger than the dataset median of approximately 870 square feet, while maintenance costs reach as high as \$4,659 despite a median value near \$733. Rather than removing these observations entirely, missing and inconsistent values were addressed through `missForest` imputation in order to preserve sample size while limiting the influence of extreme values.

Of note as well, extrapolation beyond the defined population of interest would be inappropriate, as the model was trained exclusively on co-op and condominium apartment sales

within the specified Queens zip codes during the 2016–2017 time period. Performing the former would consequently cause our predictive performance to deteriorate when applied to properties in other geographic regions, or housing markets from substantially different time periods.

2.1 Featurization

The final feature set consists of 25 predictors, drawn from both the raw dataset and externally engineered sources. Features fall into three categories: those provided directly by the raw dataset, those engineered from existing columns or external data sources, and binary missingness indicators capturing the pattern of missing data.

Provided Features

From the raw dataset the following features were used directly or with minor transformation: square footage (`sq_footage`), number of bedrooms (`num_bedrooms`), number of full bathrooms (`num_full_bathrooms`), number of total rooms (`num_total_rooms`), a binary co-op indicator (`is_coop`, derived from `coop_condo`), binary indicators for: garage existence (`garage_exists`), cats allowed (`cats_allowed`), and dogs allowed (`dogs_allowed`), kitchen type as a factor (`kitchen_type`, with levels `combo`, `eat in`, and `efficiency`), dining room type as a factor (`dining_room_type`, with levels `combo`, `formal`, and `other`), fuel type as a factor (`fuel_type`), monthly maintenance cost (`maintenance_cost`), monthly common charges (`common_charges`), walk score (`walk_score`), and number of floors in the building (`num_floors_in_building`). Community district number (`community_district_num`) was also retained from the raw data, with rare districts grouped into an “Other” category to prevent cross validation failures.

Several raw features were considered but ultimately excluded. The feature `num_half_bathrooms` was missing for 494 of 518 observations (95%) — a missingness rate so extreme that any imputed values would be essentially fabricated rather than informative, and the feature was dropped. `total_taxes` and `parking_charges` were each missing for approximately 75% of observations, making them similarly unreliable. `pct_tax_deductibl` was missing for 82% of observations. Lastly, the `model_type` column contained hundreds of inconsistent text entries ranging from “1 Bedroom” to “Gothic Tenants” to specific building names, which were incredibly noisy. This made meaningful cleaning within the scope of this project unrealistic.

Engineered Features

In an effort to enhance our dataset with contextual holistic information, five features were engineered specifically for this project, of which were not captured in the original columns.

`age_at_sale` was computed as 2016 minus `approx_year_built`, converting the raw year built into a more interpretable measure of building age at the time of sale. Through this measure, we were able to ascertain that older buildings in Queens tend to be co-ops built in the mid-20th century, while newer construction skews toward condominiums commanding premium prices.

`neighborhood` groups each observation’s zip code into one of the nine Queens regions defined in the project specification: Northeast Queens, North Queens, Central Queens, Jamaica, Northwest Queens, West Central Queens, Southeast Queens, Southwest Queens, and West Queens. Individual zip codes were deliberately excluded as a modeling feature despite their intuitive relevance to apartment pricing. With only 518 observations spread across dozens of zip codes, many zip codes had only one or two observations. During 5-fold cross validation, these rare zip codes would randomly appear exclusively in the test fold, causing OLS and YARFCART to fail with “factor has new levels” errors — a fundamental limitation of how parametric models handle unseen categorical levels. Neighborhood grouping solved this by ensuring every category had sufficient observations across all folds, while still preserving the geographic signal. The nine regions were validated against US Census Bureau ZIP Code Tabulation Areas (ZCTAs) (U.S. Census Bureau, 2016).

`mean_income` assigns each observation the approximate mean household income of its zip code for the 2016 period, sourced from the US Census Bureau’s American Community Survey (ACS) 2012-2016 5-year estimates (U.S. Census Bureau, 2016). This continuous numeric feature captures neighborhood-level wealth more precisely than the broad neighborhood grouping alone — two zip codes within the same neighborhood region may have meaningfully different income levels. For example, within North Queens, zip code 11357 (Whitestone) has a mean income of \$98,500 while 11354 (Flushing) has a mean income of only \$53,000.

`crime_rate` assigns each observation an ordinal crime safety rating for its zip code, ranging from “Extremely Safe” to “High”. Crime ratings were compiled from multiple sources: the NYC Open Data portal (complaint data historic) (NYC Open Data, 2016), the New York State Division of Criminal Justice Services (DCJS) annual crime reports (DCJS, 2017), CrimeGrade.org (2016), and NeighborhoodScout (2016). This feature was encoded as an ordered factor with six levels: Extremely Safe, Very Safe, Safe, Moderate, Moderate-High, and High.

`cpi` assigns each observation the US Consumer Price Index (All Urban Consumers) for the specific month and year of its sale, ranging from 236.9 in February 2016 to 243.6 in February 2017, sourced from the US Bureau of Labor Statistics (Bureau of Labor Statistics, 2017). Since the dataset spans a full year, the CPI feature allows the model to account for temporal variation in purchasing power across the sale period.

Missingness Indicators

Four binary missingness indicators were added to the feature set, created before imputation to capture which values were genuinely missing rather than imputed. `sq_footage_missing` flags the 60% of observations with missing square footage; `common_charges_missing` flags the 75% with missing common charges; `maintenance_cost_missing` flags the 27% with missing maintenance cost; and `num_floors_missing` flags the 21% with missing floor count. These indicators allow the model to learn from the pattern of missingness itself — for example, `common_charges` being missing strongly correlates with co-op status since co-ops use maintenance fees rather than common charges. Including these four indicators improved random forest performance (OOS RMSE \$80,422 $\rightarrow$ \$79,818) and were retained in the final model.

Feature Summaries

Continuous features range considerably — square footage spans 375 to 6,215 with a median of 870; age at sale ranges from 0 to 101 years with a median of 59; maintenance cost ranges from \$155 to \$4,659 with a median of \$733; mean income by zip ranges from \$41,500 to \$110,000 with a median of \$76,000; CPI ranges from 236.9 to 243.6 with a median of 241.0. Among categorical features, 75% of observations are co-ops, 17% have a garage, 46% allow cats, and 28% allow dogs. The most common kitchen type is eat-in (40%), followed by efficiency (44%) and combo (15%). The most common neighborhood is North Queens (112 observations), followed by West Central Queens (93) and Southwest Queens (59). Forest Hills (11375) is the most represented individual zip code with 64 observations. Crime ratings skew toward the safer end — 188 observations are rated Safe, 143 Very Safe, and only 38 High.

2.2 Errors and Missingness

Several glaringly obvious errors were identified and corrected during cleaning. For example: The `kitchen_type` column contained a value of “1955” — a year entered in the wrong field entirely — which was recoded as NA. The `dogs_allowed` column contained “yes89”, a clear typographical error recoded as “yes”. The `garage_exists` column used at least five different encodings for the same concept: “yes”, “Yes”, “eys”, “UG”, and “Underground”,

all standardized to “yes”, with NA recoded as “no” since the absence of a recorded garage most plausibly indicates no garage exists. Capitalization was inconsistent across multiple columns, with “other” and “Other” treated as separate categories — all standardized to lowercase.

Duplicate observations were also investigated. After filtering to the project population, 59 address strings appeared more than once in the dataset. Careful inspection revealed that the majority of these were legitimate separate sales of different units within the same building — large Queens co-op and condo complexes such as One Bay Club Drive, 70-25 Yellowstone Blvd, and 54-44 Little Neck Parkway appeared multiple times with different sale prices and dates, each representing a genuine distinct transaction. However, 3 observations were identified as true duplicates — identical address, sale price, and sale date — most likely the result of the same MTurk transaction being entered twice. These 3 rows were removed, reducing the dataset from 521 to 518 observations. Duplicate removal was intentionally conservative, preserving all repeated addresses with differing prices or dates as valid distinct sales.

Missingness was substantial across several features. Square footage was missing for 312 of 518 pre-imputation observations (60%), common charges for 390 (75%), maintenance cost for 140 (27%), dining room type for 115 (22%), and age at sale for 6 (1%). Binary missingness indicators were created before imputation for the four most heavily missing features and included in the final feature set, as described in Section 2.2. All remaining missingness in continuous and categorical features was then handled via missForest, a random-forest-based imputation algorithm that fits a random forest on the observed values of each feature using all other features as predictors, then predicts missing values from that forest. Unlike median or mode imputation which replaces all missing values with the same global statistic, missForest produces a unique imputed value for each missing observation based on its individual feature profile — for example, a large co-op in Forest Hills with high walk score will receive a different imputed square footage than a small condo in Jamaica. This approach is particularly well-suited to a dataset with high and potentially non-random missingness like ours.

Worth mentioning, the listing price column (`listing_price_to_nearest_1000`) was considered as a feature but dropped entirely as it exhibited 100% missingness across all 518 modeling observations. It is quite probable that the MTurk workers who recorded sale prices and those who recorded listing prices possessed entirely different sets of observations with no overlap. However, due to the substantial real-world correlation between listing price and selling price, it was momentarily considered to use listing price to approximate selling price. Given that selling price is heavily based upon what price the property was originally listed

for, in theory it would not prove far-fetched to attempt to formulate some sort of method to approximate selling price. Unfortunately, the major pitfall or “trap” regarding this is that we would essentially be fabricating data for which there is no actual in-dataset way of accounting for.

3. Modeling

3.1 Regression Tree Modeling

A single regression tree was fit using YARFCART (Kapelner, 2024) on the full dataset of 518 observations across 25 features. The tree recursively partitions the feature space by selecting the split that minimizes within-node sum of squared residuals at each step, growing until leaves are pure or a stopping criterion is met. Visualizing the top four layers reveals that the first split is on `is_coop` — immediately separating the two apartment types, which have fundamentally different pricing structures due to co-op board requirements, financing restrictions, and maintenance fee structures. Subsequent splits involve `maintenance_cost`, `num_full_bathrooms`, `common_charges`, and `neighborhood`, reflecting the dominant role of carrying costs and location in Queens apartment pricing.

The top 10 most important features by split-based variable importance are: number of full bathrooms, age at sale, co-op indicator, square footage, total rooms, maintenance cost, common charges, neighborhood, number of bedrooms, and number of floors in building. These features collectively capture the three primary drivers of apartment value: size (bathrooms, rooms, square footage), cost structure (maintenance, common charges), and location (neighborhood, floors as a proxy for building type).

3.2 Linear Modeling

An OLS linear model was fit on the full dataset. The in-sample R^2 of 0.845 indicates the linear model explains 84.5% of variance in sale price, with a residual standard error of \$72,650. The F-statistic of 60.2 on 43 and 474 degrees of freedom is highly significant ($p < 2.2e-16$), confirming the model as a whole is informative.

The most statistically significant predictors are the co-op indicator ($\beta = -\$193,500$, $p < 2e-16$), number of full bathrooms ($\beta = +\$81,113$, $p < 2e-16$), number of bedrooms ($\beta = +\$47,380$, $p < 2e-16$), maintenance cost ($\beta = +\$99$ per dollar of monthly maintenance, $p < 2e-16$), and number of floors in building ($\beta = +\$3,971$ per floor, $p < 2e-16$). The co-op coefficient is particularly notable — holding all else equal, a co-op sells for approximately

\$193,500 less than a comparable condo, reflecting the structural discount associated with cooperative ownership. Common charges ($\beta = +\$127$ per dollar, $p < 0.001$) and mean neighborhood income ($\beta = +\$1.50$ per dollar of income, $p < 0.05$) are also significant. Notably, square footage, age at sale, garage existence, cats allowed, walk score, and fuel type are not statistically significant at the 0.05 level, suggesting their effects are captured by correlated features already in the model.

A linear model may be suboptimal for prediction in this context due to its assumption of additive, linear relationships between features and sale price. Interaction effects — for example, between co-op status and maintenance cost, or between neighborhood and square footage — are not captured. Nevertheless, the competitive OOS performance of OLS relative to the random forest suggests the dominant relationships in this dataset are largely linear.

3.3 Random Forest Modeling

Random forest was chosen as the primary production model for several reasons. Random forests are non-parametric ensemble methods that make no assumptions about the functional form of the relationship between features and the outcome. By fitting 500 decision trees on bootstrap samples of the training data and averaging their predictions, the random forest dramatically reduces the variance of the single regression tree while maintaining low bias. Each tree considers only a random subset of `mtry` features at each split, decorrelating the trees and further reducing variance.

Hyperparameter Tuning

Two hyperparameters were tuned via 5-fold cross validation: `mtry` and `num_trees`. Tuning `mtry` matters because it controls the degree of decorrelation between trees — too few features per split produces highly decorrelated but individually weak trees, while too many approaches a bagged ensemble without the variance-reduction benefit of random feature selection. The default value of $\lfloor \sqrt{p} \rfloor \approx 5$ for 25 features was too conservative for this dataset. Grid search over values (3, 5, 7, 9, 11, 13, 15, 17, 19) identified `mtry = 13` as optimal. The higher-than-default optimal value suggests the model benefits from seeing more features at each split, consistent with the engineered neighborhood features adding complementary signal at each node.

Simultaneously, tuning `num_trees` matters as well as too few trees produces a high-variance ensemble that hasn't fully converged, while too many simply wastes computation without improving performance. Grid search over values (250, 500, 750, 1000) identified `num_trees = 500` as optimal. Increasing `num_trees` beyond 500 produced no further improvement,

confirming the forest had converged.

Compared to OLS, the random forest gains the ability to capture non-linear relationships and interaction effects without specification. However, as handy as it proves itself to be, it causes us to lose interpretability (there is no single coefficient table), and also requires more computation. The OOB R^2 of 0.7977 and OOB RMSE of \$79,447 serve as built-in out-of-sample estimates, consistent with the cross-validated estimates and confirming the model is not overfit. The top features by split count across all 500 trees are walk score, maintenance cost, age at sale, CPI, mean income, number of floors, square footage, total rooms, number of bedrooms, and cats allowed — broadly consistent with the regression tree’s top features, with greater weight given to the engineered neighborhood economic features.

4. Performance Results

Model	In-Sample R^2	In-Sample RMSE	OOS R^2	OOS RMSE	Normalized RMSE
Regression Tree	0.9652	\$32,941	0.5730	\$112,756	0.3615
OLS Linear	0.8511	\$68,156	0.7849	\$80,697	0.2587
Random Forest	0.9496	\$39,672	0.7950	\$79,221	0.2540

Table 1: Model Performance: In-Sample vs Out-of-Sample

The regression tree exhibits severe overfitting, its in-sample R^2 of 0.965 collapses to 0.635 out-of-sample, a stark gap of 0.330. This is normally characteristic of high-variance models: single trees memorize training data by creating highly specific splits that do not generalize. The random forest substantially mitigates this variance through bagging and random feature selection, reducing the in-sample to OOS gap to 0.159. OLS shows the smallest gap of 0.062, consistent with its parametric structure imposing regularization through the linearity assumption.

The clear winner is Random Forest — achieving the best OOS performance with RMSE of \$79,818, normalized RMSE of 0.2559, and R^2 of 0.7939, whilst narrowly outperforming OLS (RMSE \$81,510, normalized RMSE 0.2614, R^2 0.7831). The proximity of these two models though, is noteworthy. Their nearness suggests the dominant relationships in Queens apartment pricing are largely linear, and the additional flexibility of the random forest provides only marginal gains. The OOB estimates (RMSE \$79,447, R^2 0.7977) are consistent with the cross-validated estimates, confirming the validity of the OOS performance assessment. The 5-fold cross validation design ensures that every observation appears in the test set exactly once across folds, providing a reliable and nearly unbiased estimate of generalization

performance.

5. Discussion

This project demonstrates that apartment sale prices in Queens can be predicted with meaningful accuracy using a combination of property-level features, neighborhood economic indicators, and temporal economic context. The random forest achieves an out-of-sample R^2 of 0.7939, explaining nearly four-fifths of the variance in sale price using only 518 observations and 25 features. The most important predictors align with real estate intuition: carrying costs (maintenance, common charges), property characteristics (bathrooms, rooms, square footage), co-op versus condo status, and neighborhood-level economic conditions (income, walkability).

Why We Cannot Realistically Beat Zillow

The most important contextual fact about this project’s performance is that closing the gap with Zillow’s 2.3% median error is not realistically achievable within the constraints of this dataset — and understanding why illuminates what would actually be required.

Zillow’s “zestimates” are trained on tens of millions of property transactions across the entire United States, updated continuously as new sales occur. For Queens specifically, Zillow would have access to years of historical transaction data, giving it exponentially more training examples than the 518 observations available here. With more data, the model can learn neighborhood-level pricing patterns at the individual building or even individual unit level, a granularity that is simply impossible with 518 observations spread across hundreds of buildings. We are, essentially, fighting a losing battle, one in which unless we miraculously gain the data collection capabilities Zillow has acquired overnight, our odds of victory are less than slim.

Furthermore, beyond quantity, Zillow has access to fundamentally richer data. Their models incorporate satellite and street-level imagery, allowing them to assess building condition, renovation quality, and neighborhood aesthetics, signals entirely absent from our dataset and of which we have no means of retrieval. They have full transaction histories for each property, enabling price trend modeling at the individual address level. Real-time market signals such as days on market, price reductions, and listing activity, are just a few of the plentitude of indicators they have to inform them on where prices are heading, making their ignorance error much smaller than ours. For the purposes of this study, we dabbled in the process of engineering features successfully creating 5 robust ones, however this is just a

mere drop in the bucket compared to Zillow’s proprietary feature engineering which draws on decades of real estate domain expertise and billions of dollars of investment.

To highlight the disparity even more, recall that our dataset was collected by Amazon MTurk workers manually transcribing online listings, gifting us widespread missingness: 60% missingness in square footage, 75% missingness in common charges, and the complete absence of listing price data due to a structural data collection flaw, to be exact. The humbling reality is that even the most sophisticated modeling cannot compensate for fundamentally missing information. Therefore, the 25.6% normalized RMSE achieved here should be understood not as a failure of the modeling approach, but as an honest reflection of what is achievable with the available data.

Technical Challenges: Honest Remarks

The most persistent technical challenge was the rare factor level problem in cross validation. With 518 observations spread across dozens of zip codes, some zip codes appeared only once or twice. During 5-fold cross validation, these rare zip codes would randomly appear exclusively in the test fold, causing OLS and YARFCART to fail with “factor has new levels” errors. Multiple approaches were attempted, setting factor levels globally before the loop, dropping the least common zip codes, before arriving at the neighborhood grouping solution. Individual zip codes were replaced with the nine Queens regions from the project specification, ensuring every category had sufficient representation across all folds while preserving geographic signal. Statistically justifiable as neighborhood regions are more meaningful predictors than raw zip code numbers, since zip codes are arbitrary administrative boundaries while neighborhoods reflect genuine socioeconomic clustering.

A similar rare level problem arose with `dining_room_type` after duplicate removal. Prior to removing the 3 duplicate observations, the “dining area” category had enough observations to survive cross validation. After removal only 2 remained, consistently landing in the test fold. The connection between duplicate removal and the sudden appearance of this error was not immediately obvious — it required comparing pre- and post-removal category counts. The fix was collapsing “dining area” into “other”.

Moreover, merging external features also proved a bit tasking. When `merge()` was called multiple times across different attempts, R created `mean_income.x`, `mean_income.y`, and a third `mean_income` column. The fix was switching from `merge()` to `match()`, which assigns values directly without creating suffixed duplicates. A subtle bug also arose from the interaction between duplicate removal and fold index generation. After removing 3 rows the dataset shrank from 521 to 518, but fold indices had been generated for 521 rows, causing 2

NA values to appear in the response vector. The fix was regenerating fold indices after all data processing was complete.

Log Transformation: Honest Remarks

Log transformation of the outcome was explored as a potential performance improvement, given the right-skewed distribution of sale prices. Models were fit predicting $\log(\text{sale_price})$ and predictions were converted back to dollar scale via $\exp()$. However, all three models performed worse under the log transformation. OLS RMSE increased from \$81,510 to \$89,134, and RF RMSE increased from \$79,818 to \$88,530. This counterintuitive result suggests the original dollar scale is already well-suited to this dataset. The likely explanation is that while sale prices are technically right-skewed, the skew is not severe enough in this filtered population (all prices between \$55,000 and \$1,000,000) to benefit from log transformation. The $\exp()$ back-transformation also amplifies errors at the high end of the price distribution, penalizing predictions for expensive apartments more than the original scale would.

Hyperparameter Tuning

`mtry` and `num_trees` were tuned via grid search over cross-validated RMSE. The optimal `mtry` of 13 out of 25 features is notably higher than the typical default of $\lfloor p/3 \rfloor \approx 8$, suggesting the model benefits from seeing more features at each split. This is consistent with the engineered neighborhood features: `mean_income`, `crime_rate`, `neighborhood` and `cpi`, being informative enough that restricting splits to only 5 features frequently excludes them. Increasing `num_trees` beyond 500 produced no further improvement, confirming forest convergence.

Limitations and Future Work

Several limitations warrant acknowledgment. The dataset’s small size constrains model complexity and limits neighborhood-level representation. The extreme missingness in key features means that even with `missForest`’s sophisticated imputation approach, heavily imputed values may introduce noise. Future extensions could incorporate NYC PLUTO building data, census tract demographics, subway proximity, or a larger multi-year training dataset. Interaction terms in OLS (e.g. `is_coop * maintenance_cost`) could improve linear model performance. Gradient boosting methods such as XGBoost may outperform random forests on this tabular dataset.

Regarding production readiness, with a normalized RMSE of 0.2559, the typical prediction error is about 25.6% of the mean sale price. In short, this is not production ready for real-world deployment, which values small error rates in trying to account for various scenarios.

However, within the constraints of this dataset the modeling framework is sound, the cross validation methodology is rigorous, and the feature engineering demonstrates genuine domain signal. It is within the realm of possibility that with access to richer data sources, our approach developed could form the foundation of a competitive apartment pricing tool.

To end this paper, we muse on the words of American statistician W. Edwards Deming, who once remarked, “In God we trust. All others must bring data.” Perhaps, in the conquest of striving to beat Zillow the true lesson to be learned was that effective prediction is not achieved through intuition alone, but through careful data collection, thoughtful feature engineering, rigorous modeling, and continual evaluation. Although no predictive model is perfect, this analysis illustrates how statistical learning methods can transform raw housing data into informed and actionable predictions.

Acknowledgments

I would like to express my gratitude to Professor Kapelner for his feedback, and support throughout the completion of this project. Also, I would like to thank the NYC Open Data portal, the New York State Division of Criminal Justice Services, CrimeGrade.org, NeighborhoodScout, the US Census Bureau, and the US Bureau of Labor Statistics for making the data used in the engineered features publicly available.

References

- Bureau of Labor Statistics, U.S. Department of Labor. (2017). *Consumer Price Index for All Urban Consumers (CPI-U)*. U.S. Bureau of Labor Statistics. <https://www.bls.gov/cpi/>
- CrimeGrade.org. (2016). *Queens, NY neighborhood crime grades*. <https://crimegrade.org>
- Kapelner, A. (2024). *YARF: Yet Another Random Forest* [R package]. GitHub. <https://github.com/kapelner/YARF>
- Multiple Listing Service of Long Island (MLSI). (2017). *Queens apartment sales data 2016-2017* [Dataset]. Retrieved via Amazon Mechanical Turk.
- New York City Open Data. (2016). *NYPD complaint data historic* [Dataset]. NYC Open Data. <https://data.cityofnewyork.us>
- New York State Division of Criminal Justice Services. (2017). *Index crime by county and agency: 2016* [Report]. DCJS. <https://www.criminaljustice.ny.gov>

NeighborhoodScout. (2016). *Queens, NY neighborhood crime and safety data*. <https://www.neighborhoodscout.com>

U.S. Census Bureau. (2016). *American Community Survey 5-year estimates, 2012-2016* [Dataset]. U.S. Census Bureau. <https://www.census.gov/programs-surveys/acs>

U.S. Census Bureau. (2016). *ZIP Code Tabulation Areas (ZCTAs)* [Dataset]. U.S. Census Bureau. <https://www.census.gov/programs-surveys/geography/guidance/geo-areas/zctas.html>

Zillow. (2020). *Zestimate accuracy*. Zillow Research. <https://www.zillow.com/research/zestimate-accuracy>

Code Appendix

All code used in this project is reproduced below.

```
## Core Libraries
library(tidyverse)
library(magrittr)

install.packages("stringr")
library(stringr)

## Modeling
install.packages("rJava")
install.packages("devtools")
devtools::install_github("kapelner/YARF/YARFJARs", ref = "dev")
devtools::install_github("kapelner/YARF/YARF", ref = "dev")

options(java.parameters = "-Xmx4000m")
install.packages("pacman")
library(pacman)
pacman::p_load(rJava)
library(rJava)
.jinit()

if (!pacman::p_isinstalled(YARF)){
  pacman::p_install_gh("kapelner/YARF/YARFJARs", ref = "dev")
  pacman::p_install_gh("kapelner/YARF/YARF", ref = "dev", force = TRUE)
}
pacman::p_load(YARF)

library(YARF)          # regression tree + random forest
library(rpart)         # fallback tree visualization
library(rpart.plot)
library(knitr)
library(kableExtra)
library(missForest)
library(naniar)

set.seed(342)

# ---- Loading Our Data ----
housing <- read.csv("housing_data_2016_2017.csv",
                   stringsAsFactors = FALSE)

dim(housing)
```



```

names(housing)
cat("Rows:", nrow(housing), "\n")
cat("Columns:", ncol(housing), "\n")

# ---- Understanding Our Data ----
## since coop_condo is our apartment type filter,
## we first check what the apartment type values are
table(housing$coop_condo, useNA = "always")

## Let's look at how our zip codes are stored
head(housing$full_address_or_zip_code, 20)

## Check sale prices
summary(housing$sale_price)
head(housing$sale_price, 20)

## Examine rest of data
table(housing$garage_exists, useNA = "always")
## "yes", "Yes", "eys", "Underground", "UG", and "1"
## most likely all mean the same thing.
## "NA" probably means no garage

table(housing$cats_allowed, useNA = "always")
## cats_allowed has "y" and "yes" as separate values

table(housing$dogs_allowed, useNA = "always")
## dogs_allowed has "yes89" and "yes" as separate values,
## we'll combine them as "yes"

table(housing$fuel_type, useNA = "always")
## fuel_type has "other" and "Other" as separate values

table(housing$kitchen_type, useNA = "always")
## kitchen_type has "combo" and "Combo" as separate values,
## along with: "eat in", "Eat In" and "eatin" and also
## "efficiemcy", "efficiency", "efficiency kitchen",
## "efficiency kitche" and "efficiency ktchen"

table(housing$dining_room_type, useNA = "always")
table(housing$community_district_num, useNA = "always")

summary(housing$total_taxes)
summary(housing$parking_charges)

```

```

summary(housing$num_floors_in_building)
summary(housing$sq_footage)
summary(housing$num_bedrooms)
summary(housing$num_full_bathrooms)
summary(housing$approx_year_built)
summary(housing$walk_score)
summary(housing$maintenance_cost)
summary(housing$common_charges)
summary(housing$listing_price_to_nearest_1000)
summary(housing$num_half_bathrooms)
summary(housing$pct_tax_deductibl)

# ---- Cleaning Our Data ----
## our zip codes are stuck in address string,
## so we need to extract them
housing$zipcode <- as.integer(stringr::str_extract(
  housing$full_address_or_zip_code, "\\d{5}"))
## taking anywhere we see 5 digits in a row

## Change sale price to numeric from string
housing$sale_price <- as.numeric(
  gsub("$,\\s", "", housing$sale_price))
## gsub finds every match of a pattern in a string
## and replaces it - finding "$" and "," and removing it
## then turning final result into a number

## Change listing price to numeric from string
housing$listing_price_to_nearest_1000 <- as.numeric(
  gsub("$,\\s", "", housing$listing_price_to_nearest_1000))

## Change total taxes to numeric from string
housing$total_taxes <- as.numeric(
  gsub("$,\\s", "", housing$total_taxes))

## Change parking charges to numeric from string
housing$parking_charges <- as.numeric(
  gsub("$,\\s", "", housing$parking_charges))

## Change maintenance cost to numeric
housing$maintenance_cost <- as.numeric(
  gsub("$,\\s", "", housing$maintenance_cost))

## Change common charges to numeric

```

```

housing$common_charges <- as.numeric(
  gsub("$,\\s", "", housing$common_charges))

## Standardizing cats_allowed
housing$cats_allowed <- ifelse(
  housing$cats_allowed == "y", "yes", housing$cats_allowed)
housing$cats_allowed <- tolower(housing$cats_allowed)
## Replaces y with yes, and for entries with "Yes" or "No",
## changes to standard "yes" or "no"

## Standardizing dogs_allowed
housing$dogs_allowed <- ifelse(
  housing$dogs_allowed == "yes89", "yes", housing$dogs_allowed)
housing$dogs_allowed <- tolower(housing$dogs_allowed)

## Standardizing fuel_type
housing$fuel_type <- tolower(housing$fuel_type)

## Standardizing kitchen type
housing$kitchen_type <- tolower(housing$kitchen_type)
housing$kitchen_type <- ifelse(housing$kitchen_type %in%
  c("efficiemcy", "efficiency kitchen", "efficiency kitche",
    "efficiency ktchen", "efficiency"),
  "efficiency", housing$kitchen_type)
housing$kitchen_type <- ifelse(
  housing$kitchen_type == "1955", NA, housing$kitchen_type)
housing$kitchen_type <- ifelse(
  housing$kitchen_type %in% c("combo", "Combo"),
  "combo", housing$kitchen_type)
housing$kitchen_type <- ifelse(
  housing$kitchen_type == "eatin", "eat in", housing$kitchen_type)

## Standardizing dining_room_type
housing$dining_room_type <- as.character(housing$dining_room_type)
housing$dining_room_type <- ifelse(
  housing$dining_room_type == "dining area",
  "other", housing$dining_room_type)
## Changes the 2 entries that say "dining area" to "other"

## Standardizing garage exists
housing$garage_exists <- tolower(housing$garage_exists)
housing$garage_exists <- ifelse(housing$garage_exists %in%
  c("yes", "eys", "ug", "underground", "1"),

```

```

    "yes", housing$garage_exists)
housing$garage_exists <- ifelse(
  is.na(housing$garage_exists), "no", housing$garage_exists)

summary(housing$sale_price)
summary(housing$maintenance_cost)
summary(housing$common_charges)
summary(housing$listing_price_to_nearest_1000)
summary(housing$total_taxes)
summary(housing$parking_charges)
summary(housing$num_floors_in_building)
table(housing$cats_allowed, useNA = "always")
table(housing$dogs_allowed, useNA = "always")
table(housing$fuel_type, useNA = "always")
table(housing$kitchen_type, useNA = "always")
table(housing$dining_room_type, useNA = "always")
table(housing$garage_exists, useNA = "always")
table(housing$zipcode, useNA = "always")
table(housing$community_district_num, useNA = "always")

# ---- Filter to Project Population ----
valid_zipcodes <- c(
  11361, 11362, 11363, 11364, 11354, 11355, 11356, 11357,
  11358, 11359, 11360, 11365, 11366, 11367, 11412, 11423,
  11432, 11433, 11434, 11435, 11436, 11101, 11102, 11103,
  11104, 11105, 11106, 11374, 11375, 11379, 11385, 11004,
  11005, 11411, 11413, 11422, 11426, 11427, 11428, 11429,
  11414, 11415, 11416, 11417, 11418, 11419, 11420, 11421,
  11368, 11369, 11370, 11372, 11373, 11377, 11378)

## Formatting dataset as how project specified
housing <- housing %>% filter(
  zipcode %in% valid_zipcodes,
  coop_condo %in% c("co-op", "condo"),
  !is.na(sale_price),
  sale_price <= 1000000,
  sale_price > 0)

## Now lets check if we have any duplicate addresses
## in our filtered dataset (same address listed with same price,
## not referring to multiple units in a building)
dup_addresses <- housing$full_address_or_zip_code[
  duplicated(housing$full_address_or_zip_code)]

```

```

cat("Number of duplicate addresses:", length(dup_addresses), "\n")

## View duplicates with sale price and date
result <- housing[housing$full_address_or_zip_code
                  %in% dup_addresses,
                  c("full_address_or_zip_code", "sale_price", "date_of_sale")]
result <- result[order(result$full_address_or_zip_code), ]
print(result)

## Remove true duplicates (same address, same price, same date)
housing <- housing[!duplicated(paste(
  housing$full_address_or_zip_code,
  housing$sale_price,
  housing$date_of_sale)), ]

## reset row numbers
rownames(housing) <- NULL

cat("Rows after removing duplicates:", nrow(housing), "\n")
dim(housing)
summary(housing$sale_price)

# ---- Featurization ----
housing <- housing %>% mutate(
  age_at_sale = 2016 - approx_year_built,
  ## converting yes/no features into binary
  ## and categorizing others
  garage_exists = ifelse(garage_exists == "yes", 1, 0),
  is_coop = ifelse(coop_condo == "co-op", 1, 0),
  cats_allowed = ifelse(cats_allowed == "yes", 1, 0),
  dogs_allowed = ifelse(dogs_allowed == "yes", 1, 0),
  kitchen_type = as.factor(kitchen_type),
  dining_room_type = as.factor(dining_room_type),
  fuel_type = as.factor(fuel_type))

housing$community_district_num <- as.factor(
  housing$community_district_num)

## Now, in addition to the features provided by our dataset,
## to achieve the most accurate model possible, we will
## introduce some external features. The values were extracted
## via national data collection during the time period of
## 2016-2017 relative to each zip code.

```

```

## Mean Income by Zipcode (ACS 2012-2016)
mean_income_by_zip <- data.frame(
  zips = c(11004, 11101, 11102, 11103, 11104, 11105, 11106,
    11109, 11354, 11355, 11356, 11357, 11358, 11360, 11361,
    11362, 11363, 11364, 11365, 11366, 11367, 11368, 11369,
    11370, 11372, 11373, 11374, 11375, 11377, 11378, 11379,
    11385, 11411, 11412, 11413, 11414, 11415, 11416, 11417,
    11418, 11419, 11420, 11421, 11422, 11423, 11426, 11427,
    11428, 11429, 11432, 11433, 11434, 11435, 11436, 11691,
    11692, 11693, 11694, 11697),
  mean_income = c(
    91500, 67000, 62000, 64000, 60000, 80000, 58500, 125000,
    53000, 50000, 76000, 98500, 85000, 88500, 86000, 100000,
    110000, 82000, 73000, 78000, 68000, 41500, 49500, 61000,
    55000, 48000, 66000, 80000, 59000, 73000, 85000, 57000,
    78000, 74000, 70000, 85000, 66000, 57000, 59000, 55000,
    53000, 59000, 57000, 78000, 67000, 90000, 78000, 76000,
    74000, 53000, 46000, 48000, 44000, 53000, 40000, 47000,
    69000, 86000, 122500))

housing$mean_income <- mean_income_by_zip$mean_income[
  match(housing$zipcode, mean_income_by_zip$zips)]

## Check it worked
sum(is.na(housing$mean_income))
summary(housing$mean_income)

# Grouping community districts
## Several districts have only 1 observation: districts 3, 4,
## 8, 16, 19, 20, 23. These will randomly end up only in the
## test fold in later sections, resulting in errors.
housing$community_district_num <- as.character(
  housing$community_district_num)
housing$community_district_num <- ifelse(
  housing$community_district_num %in%
    c("24", "25", "26", "27", "28", "29", "30"),
  housing$community_district_num, "Other")
housing$community_district_num <- as.factor(
  housing$community_district_num)
table(housing$community_district_num)

## Crime Ranking by Zipcode

```

```

## Sources: NYC Open Data, DCJS, CrimeGrade.org, NeighborhoodScout
crime_by_zip <- data.frame(
  zips = c(11004, 11101, 11102, 11103, 11104, 11105, 11106,
    11109, 11354, 11355, 11356, 11357, 11358, 11360, 11361,
    11362, 11363, 11364, 11365, 11366, 11367, 11368, 11369,
    11370, 11372, 11373, 11374, 11375, 11377, 11378, 11379,
    11385, 11411, 11412, 11413, 11414, 11415, 11416, 11417,
    11418, 11419, 11420, 11421, 11422, 11423, 11426, 11427,
    11428, 11429, 11432, 11433, 11434, 11435, 11436, 11691,
    11692, 11693, 11694, 11697),
  crime_rate = factor(c(
    "Very Safe", "High", "Moderate", "Moderate", "Moderate",
    "Safe", "Moderate", "Safe", "Moderate", "Moderate", "Safe",
    "Very Safe", "Very Safe", "Very Safe", "Very Safe",
    "Very Safe", "Extremely Safe", "Very Safe", "Safe", "Safe",
    "Safe", "High", "Moderate-High", "Moderate-High",
    "Moderate-High", "High", "Safe", "Safe", "Moderate", "Safe",
    "Very Safe", "Moderate-High", "Safe", "Safe", "Moderate",
    "Safe", "Safe", "Moderate", "Moderate", "Moderate",
    "Moderate", "Moderate", "Moderate", "Safe", "Moderate",
    "Very Safe", "Safe", "Safe", "Safe", "Moderate-High",
    "High", "High", "High", "Moderate-High", "High",
    "Moderate-High", "Moderate", "Safe", "Extremely Safe"),
  levels = c("Extremely Safe", "Very Safe", "Safe", "Moderate",
    "Moderate-High", "High"))

housing$crime_rate <- crime_by_zip$crime_rate[
  match(housing$zipcode, crime_by_zip$zips)]

sum(is.na(housing$crime_rate))
table(housing$crime_rate)

## Neighborhood grouping per project specification
housing$neighborhood <- ifelse(
  housing$zipcode %in% c(11361, 11362, 11363, 11364),
  "Northeast Queens",
  ifelse(housing$zipcode %in%
    c(11354, 11355, 11356, 11357, 11358, 11359, 11360),
  "North Queens",
  ifelse(housing$zipcode %in% c(11365, 11366, 11367),
  "Central Queens",
  ifelse(housing$zipcode %in%
    c(11412, 11423, 11432, 11433, 11434, 11435, 11436),

```

```

    "Jamaica",
    ifelse(housing$zipcode %in%
      c(11101, 11102, 11103, 11104, 11105, 11106),
    "Northwest Queens",
    ifelse(housing$zipcode %in% c(11374, 11375, 11379, 11385),
    "West Central Queens",
    ifelse(housing$zipcode %in%
      c(11004, 11005, 11411, 11413, 11422, 11426,
        11427, 11428, 11429),
    "Southeast Queens",
    ifelse(housing$zipcode %in%
      c(11414, 11415, 11416, 11417, 11418, 11419, 11420, 11421),
    "Southwest Queens",
    ifelse(housing$zipcode %in%
      c(11368, 11369, 11370, 11372, 11373, 11377, 11378),
    "West Queens", NA)))))))))
housing$neighborhood <- as.factor(housing$neighborhood)
table(housing$neighborhood)
sum(is.na(housing$neighborhood))

## Monthly Inflation Rate (CPI)
## Extract month and year from date_of_sale
housing$sale_month <- as.integer(format(
  as.Date(housing$date_of_sale, "%m/%d/%Y"), "%m"))
housing$sale_year <- as.integer(format(
  as.Date(housing$date_of_sale, "%m/%d/%Y"), "%Y"))

## US Monthly CPI (All Urban Consumers, 2016-2017)
## Source: Bureau of Labor Statistics
cpi_data <- data.frame(
  year = c(2016, 2016, 2016, 2016, 2016, 2016, 2016,
    2016, 2016, 2016, 2016, 2017, 2017),
  month = c(2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2),
  cpi = c(236.9, 238.1, 239.3, 240.2, 241.0, 240.6,
    240.8, 241.4, 241.7, 241.4, 241.4, 242.8, 243.6))

housing$cpi <- cpi_data$cpi[
  match(paste(housing$sale_year, housing$sale_month),
    paste(cpi_data$year, cpi_data$month))]

sum(is.na(housing$cpi))
summary(housing$cpi)

```



```

## ---- Missingness Indicators ----
## Added before imputation so we capture which values
## were truly missing. sq_footage and common_charges have
## very high missingness (60% and 75%), which may itself
## be informative e.g. common_charges missing often
## indicates a co-op rather than a condo.
housing$sq_footage_missing <- as.integer(
  is.na(housing$sq_footage))
housing$common_charges_missing <- as.integer(
  is.na(housing$common_charges))
housing$maintenance_cost_missing <- as.integer(
  is.na(housing$maintenance_cost))
housing$num_floors_missing <- as.integer(
  is.na(housing$num_floors_in_building))

cat("sq_footage missing:", sum(housing$sq_footage_missing), "\n")
cat("common_charges missing:",
    sum(housing$common_charges_missing), "\n")
cat("maintenance_cost missing:",
    sum(housing$maintenance_cost_missing), "\n")
cat("num_floors missing:", sum(housing$num_floors_missing), "\n")

## Selecting our total final features
features <- c(
  "sq_footage",
  "age_at_sale",
  "num_bedrooms",
  "num_full_bathrooms",
  "num_total_rooms",
  "is_coop",
  "garage_exists",
  "cats_allowed",
  "dogs_allowed",
  "kitchen_type",
  "dining_room_type",
  "fuel_type",
  "maintenance_cost",
  "common_charges",
  "walk_score",
  "mean_income",
  "crime_rate",
  "neighborhood",
  "cpi",

```

```

    "num_floors_in_building",
    "community_district_num",
    "sq_footage_missing",
    "common_charges_missing",
    "maintenance_cost_missing",
    "num_floors_missing")

## Of note listing price was considered to be used as a feature
## originally to help estimate sale price as an approximation.
## However:
both <- sum(!is.na(housing$sale_price) &
            !is.na(housing$listing_price_to_nearest_1000))
cat("Rows with both sale price and listing price:", both, "\n")
## Therefore we will instead drop listing_price as a feature
## as there is no overlap between the two variables.

cat("Number of features:", length(features), "\n")
dim(housing[, features])
## Our matrix has 518 observations (rows), 25 features (columns)
summary(housing[, features])

## A few things to note: sq_footage and common_charges have
## 312 and 390 NAs respectively (over half of our observations),
## maintenance_cost has 140 NAs, kitchen_type 7 NAs,
## dining_room_type 115 NAs, age_at_sale 6 NAs

# ---- Imputation via missForest ----
library(missForest)

## Select only our feature columns for imputation
housing_for_imputation <- housing[, features]

## Run missForest - fits a random forest for each feature
## using all other features as predictors to impute
## missing values, producing unique imputed values
## per observation rather than a single global statistic
imputed <- missForest(housing_for_imputation)

## Replace feature columns with imputed values
housing[, features] <- imputed$ximp

## Check OOB imputation error
cat("missForest OOB imputation error:\n")

```

```

print(imputed$OOBError)

## Confirm no NAs remain
cat("NAs remaining after missForest:",
    sum(is.na(housing[, features])), "\n")

# ---- K-Fold Cross Validation ----
## Final cross validation with all three models using
## tuned parameters.
## mtry = 13 and num_trees = 500 were determined via tuning
## (see tuning section below)

set.seed(342)
n <- nrow(housing)
k <- 5
fold_idx <- sample(rep(1:k, length.out = n))

cat("n:", n, "\n")
cat("Fold sizes:\n")
print(table(fold_idx))

ols_rmse_folds <- c()
ols_r2_folds <- c()
tree_rmse_folds <- c()
tree_r2_folds <- c()
rf_rmse_folds <- c()
rf_r2_folds <- c()

for (fold in 1:k) {

  test_idx <- which(fold_idx == fold)
  train_idx <- which(fold_idx != fold)

  X_train <- housing[train_idx, features]
  y_train <- housing$sale_price[train_idx]
  X_test <- housing[test_idx, features]
  y_test <- housing$sale_price[test_idx]

  # --- OLS ---
  ols_model <- lm(sale_price ~ .,
    data = cbind(X_train, sale_price = y_train))
  ols_preds <- predict(ols_model, X_test)
  ols_rmse_folds[fold] <- sqrt(mean((y_test - ols_preds)^2))

```

```

ols_r2_folds[fold] <- 1 - sum((y_test - ols_preds)^2) /
  sum((y_test - mean(y_test))^2)

# --- Regression Tree ---
tree_model <- YARFCART(X_train, y_train)
tree_preds <- predict(tree_model, X_test)
tree_rmse_folds[fold] <- sqrt(mean((y_test - tree_preds)^2))
tree_r2_folds[fold] <- 1 - sum((y_test - tree_preds)^2) /
  sum((y_test - mean(y_test))^2)

# --- Random Forest (tuned) ---
rf_model <- YARF(X_train, y_train, num_trees = 500, mtry = 13)
rf_preds <- predict(rf_model, X_test)
rf_rmse_folds[fold] <- sqrt(mean((y_test - rf_preds)^2))
rf_r2_folds[fold] <- 1 - sum((y_test - rf_preds)^2) /
  sum((y_test - mean(y_test))^2)

cat("Fold", fold, "complete\n")
}

cat("\n--- 5-Fold Cross Validation Results ---\n")
cat("OLS RMSE:", round(mean(ols_rmse_folds), 2),
    "R2:", round(mean(ols_r2_folds), 4), "\n")
cat("Tree RMSE:", round(mean(tree_rmse_folds), 2),
    "R2:", round(mean(tree_r2_folds), 4), "\n")
cat("RF RMSE:", round(mean(rf_rmse_folds), 2),
    "R2:", round(mean(rf_r2_folds), 4), "\n")

mean_sale_price <- mean(housing$sale_price)

cat("\n--- Normalized RMSE (as proportion of mean sale price) ---\n")
cat("Mean sale price:", round(mean_sale_price, 2), "\n")
cat("OLS Normalized RMSE:",
    round(mean(ols_rmse_folds) / mean_sale_price, 4), "\n")
cat("Tree Normalized RMSE:",
    round(mean(tree_rmse_folds) / mean_sale_price, 4), "\n")
cat("RF Normalized RMSE:",
    round(mean(rf_rmse_folds) / mean_sale_price, 4), "\n")

# ---- Tuning Section for mtry and ntrees ----
## Mtry
n <- nrow(housing)
k <- 5

```

```

fold_idx <- sample(rep(1:k, length.out = n))

mtry_values <- c(9, 11, 13, 15, 17, 19)
mtry_rmse <- c()

for (m in mtry_values) {
  rmse_folds <- c()

  for (fold in 1:k) {
    test_idx <- which(fold_idx == fold)
    train_idx <- which(fold_idx != fold)

    X_train <- housing[train_idx, features]
    y_train <- housing$sale_price[train_idx]
    X_test <- housing[test_idx, features]
    y_test <- housing$sale_price[test_idx]

    rf <- YARF(X_train, y_train, num_trees = 500, mtry = m)
    preds <- predict(rf, X_test)
    rmse_folds[fold] <- sqrt(mean((y_test - preds)^2))
  }

  mtry_rmse <- c(mtry_rmse, mean(rmse_folds))
  cat("mtry =", m, "RMSE:", round(mean(rmse_folds), 2), "\n")
}

optimal_mtry <- mtry_values[which.min(mtry_rmse)]
cat("\nBest mtry:", optimal_mtry, "\n")

## Ntrees
ntree_values <- c(250, 500, 750, 1000)
ntree_rmse <- c()

for (nt in ntree_values) {
  rmse_folds <- c()

  for (fold in 1:k) {
    test_idx <- which(fold_idx == fold)
    train_idx <- which(fold_idx != fold)

    X_train <- housing[train_idx, features]
    y_train <- housing$sale_price[train_idx]
    X_test <- housing[test_idx, features]

```

```

y_test <- housing$sale_price[test_idx]

rf <- YARF(X_train, y_train,
           num_trees = nt, mtry = optimal_mtry)
preds <- predict(rf, X_test)
rmse_folds[fold] <- sqrt(mean((y_test - preds)^2))
}

ntree_rmse <- c(ntree_rmse, mean(rmse_folds))
cat("num_trees =", nt, "RMSE:", round(mean(rmse_folds), 2), "\n")
}

optimal_ntrees <- ntree_values[which.min(ntree_rmse)]
cat("\nBest num_trees:", optimal_ntrees, "\n")

# ---- Final Fit on Full Dataset ----
X_full <- housing[, features]
y_full <- housing$sale_price

## Final OLS
final_ols <- lm(sale_price ~ .,
               data = cbind(X_full, sale_price = y_full))
summary(final_ols)

## Final Regression Tree
final_tree <- YARFCART(X_full, y_full)

## Final Random Forest
final_rf <- YARF(X_full, y_full,
               num_trees = 500, mtry = 13, calculate_oob_error = TRUE)

cat("RF OOB RMSE:", round(final_rf$rmse_oob, 2), "\n")
cat("RF OOB R2: ", round(final_rf$pseudo_rsquared_oob, 4), "\n")

# ---- Visualizing Regression Tree for Interpretation ----
illustrate_trees(final_tree, max_depth = 4, open_file = TRUE)

tree_rpart <- rpart(sale_price ~ .,
                   data = cbind(X_full, sale_price = y_full),
                   control = rpart.control(maxdepth = 6))

var_imp <- tree_rpart$variable.importance
var_imp_df <- data.frame(

```

```

Feature      = names(var_imp),
Importance   = round(var_imp, 2)) %>% arrange(desc(Importance))

kable(head(var_imp_df, 10),
  caption = "Top 10 Features by Variable Importance (Tree)" %>%
  kable_styling(font_size = 9)

# ---- Variable Importance for Random Forest ----
var_counts <- query_variable_counts(final_rf)
var_counts_df <- data.frame(
  feature = names(var_counts),
  count   = as.numeric(var_counts)) %>% arrange(desc(count))

## Bar Plot
barplot(head(var_counts_df$count, 10),
  names.arg = head(var_counts_df$feature, 10),
  las      = 2,
  main     = "Random Forest Variable Importance (Split Count)",
  ylab     = "Number of Splits",
  col      = "steelblue",
  cex.names = 0.7)

## Table
kable(head(var_counts_df, 10),
  caption = "Top 10 Features by Split Count (Random Forest)" %>%
  kable_styling(font_size = 9)

# ---- Final Performance Results ----
results <- data.frame(
  Model = c("Regression Tree", "OLS Linear", "Random Forest"),
  'In-Sample R2' = c(
    round(1 - sum((y_full - predict(final_tree, X_full))^2) /
      sum((y_full - mean(y_full))^2), 4),
    round(summary(final_ols)$r.squared, 4),
    round(1 - sum((y_full - predict(final_rf, X_full))^2) /
      sum((y_full - mean(y_full))^2), 4)),
  'In-Sample RMSE' = c(
    round(sqrt(mean((y_full - predict(final_tree, X_full))^2)), 2),
    round(sqrt(mean((y_full - predict(final_ols, X_full))^2)), 2),
    round(sqrt(mean((y_full - predict(final_rf, X_full))^2)), 2)),
  'OOS R2' = c(
    round(mean(tree_r2_folds), 4),
    round(mean(ols_r2_folds), 4),

```

```

    round(mean(rf_r2_folds), 4)),
'OOS RMSE' = c(
    round(mean(tree_rmse_folds), 2),
    round(mean(ols_rmse_folds), 2),
    round(mean(rf_rmse_folds), 2)),
check.names = FALSE)

kable(results,
  caption = "Model Performance: In-Sample vs Out-of-Sample",
  align   = "lcccc") %>%
kable_styling(full_width = FALSE)

```